ISLAMIC UNIVERSITY OF TECHNOLOGY

A subsidiary organ of the Organisation of Islamic Cooperation (OIC)

Board Bazar, Gazipur, Bangladesh

Department of Computer Science and Engineering

FINAL PROJECT REPORT

KIDSCRIBE

A tablet app that teaches children aged five to eight to write by hand

SUBMITTED BY · GROUP 18

Hasan Mahi	230042129
Zuhaaer Mustafa Sadab	230042132
Md Shahib Al Kabir	230042157

COURSE

Software Project Lab II

SWE 4404

Academic year 2024–25

Source code: github.com/Kabirohi/kidscribe-spl2-project

1 November 2026

Abstract

Children in their first years of school learn to write mostly with pencil and paper, while the learning apps they use on tablets are built around tapping and dragging. This report describes KidScribe, an Android app we developed for Software Project Lab II that uses a stylus to teach handwriting to children aged five to eight. Practice is arranged in five stages, from basic strokes through the capital letters, short words and simple sentences. Each letter is first traced over numbered points and then written again without any guide. The free-hand attempt is read by Google's on-device ML Kit handwriting model and compared with a model shape by our own geometric checker, which also verifies stroke order and direction and tells the child which part of the letter needs work.

Parents and teachers use the same app to manage child profiles and classes, set challenges, tasks and timed tests with deadlines, grade work, leave feedback and export progress reports. A parent can photograph a child's toy to turn its name into a practice word. The app follows the BLoC architecture in Flutter, stores data on the tablet first and syncs it through Firebase when asked.

All 33 user stories in the final requirements are implemented. Of the 80 unit tests in the specification, 58 are automated through 129 Dart test cases that all pass; the other 22 require a physical device and are run manually. The report follows the project through its requirements, design and implementation sessions and closes with its limitations and the work we would do next.

KEYWORDS handwriting, early education, Flutter, BLoC, digital ink, on-device machine learning, Firebase

Contents

1 Executive Summary 4
1.1 The problem and our approach 4
1.2 Outcome 4

2 Problem Domain and Background 5
2.1 Domain, stakeholders and motivation 5
2.2 Computing and software engineering concepts applied 5
2.3 Specialist domain knowledge 6

3 Literature and Existing Solutions 7
3.1 Research on handwriting and learning 7
3.2 Comparable applications 7
3.3 Where KidScribe fits 7

4 Requirements (SRS Session) 9
4.1 Writing and tracing the requirements 9
4.2 Non-functional requirements 10
4.3 Context and sustainable development 10
4.4 Changes after the SRS review 10

5 System Design (Design Session) 12
5.1 Architecture 12
5.2 Components and data model 12
5.3 Alternatives and trade-offs 13
5.4 Design review and resulting changes 14

6 Implementation (Implementation Session) 15
6.1 Repository structure 15
6.2 Communication between components 15
6.3 End-to-end example: a free-hand letter A 16
6.4 Classes, methods and code size 16
6.5 Changes in the final phase 17

7 Tools, Version Control, Testing and CI/CD 19
7.1 Tools and alternatives 19
7.2 Version control 19
7.3 Testing 20
7.4 Planned CI/CD pipeline 21

8 Professional, Ethical and Legal Considerations 22
8.1 Licences of third-party packages 22
8.2 Artwork, fonts and sound 22
8.3 Privacy, safety and ethics 22
8.4 Professional responsibility 23

9 Project Management and Feasibility 24

9.1 Milestones 24
9.2 Tracking the work 24
9.3 Technical problems and their resolution 24
9.4 Feasibility and cost 25

10 Teamwork and Individual Contributions 26
10.1 Responsibilities 26
10.2 Coordination between modules 26
10.3 Handling disagreements 26

11 Results, Limitations and Future Work 27
11.1 Status of each requirement 27
11.2 Known limitations 27
11.3 Future work 28
11.4 Reflection 28

12 Glossary and References 28
12.1 Glossary 28
12.2 References 29

Course outcome map

The course outcome questions are answered within the chapters where the relevant work is described. The map below shows where to look.

Table 0.1 Course outcomes and the chapters that address them

CO	Chapters	Evidence provided
CO1	2.2, 2.3, 6.4	Computing, software engineering and domain knowledge applied, with the code that uses it
CO3	5, 6	Design decisions, their alternatives, and how the design was implemented
CO4	7, 8	Tools, version control, testing, CI/CD, licensing, privacy and ethics
CO6	9, 10, 11	Planning, teamwork, results, limitations and reflection

1 Executive Summary

KidScribe is an Android tablet app that helps children between five and eight learn to write by hand. The child writes with a stylus directly on the screen, and the app checks each attempt, gives a score and suggests one thing to improve. Learning is arranged in five stages that open one after another: basic strokes, the capital letters A to Z, short words, simple sentences, and a grown-up area for sync and the paid plan. Three games (Letter Match, Letter Ninja and Free Writing), badges and a daily streak are there to keep practice regular.

Adults use the same app through different screens. Parents add profiles for their children, follow weekly progress, set small writing challenges and schedule a daily reminder. They can also photograph one of their child's toys; the app suggests a name for it and adds it as a practice word. Teachers create classes, see at a glance which students are falling behind, hand out tasks and timed tests, grade them against an answer key, leave written feedback and export a report before parent meetings.

1.1 The problem and our approach

Most learning apps for this age group are built around tapping. A child taps the right letter, drags a picture into place or follows a path with a finger. These activities help a child recognise letters, but forming a letter is a motor skill that needs its own practice. Apps that do include handwriting usually stop at tracing and accept any attempt that stays near the dotted line, so a child can finish every level without once writing a letter unaided.

KidScribe splits every letter into two parts. The child first traces it over numbered points, which shows the order and direction of each stroke. The guide then disappears and the child writes the same letter on a blank page. Google's ML Kit handwriting model, running on the tablet, reads the free-hand attempt, and our own geometric check compares it with the model shape. The letter counts only when both parts have passed. When an attempt falls short, the app names the part that needs work, such as the top of the letter or a stroke drawn the wrong way, rather than showing a bare number.

The second problem is visibility. Parents rarely know what was practised during the day, and a teacher with thirty pupils cannot watch each of them write. Since every attempt is saved under the child who made it, the same data can be shown to the responsible adult, and the class view flags students who are well behind the rest.

1.2 Outcome

All 33 user stories in the final SRS, spread over 24 functional requirements, are implemented. The app is written in Dart with Flutter using the BLoC pattern, passes `flutter analyze` without warnings and builds an Android APK. Of the 80 unit tests listed in the SRS, 58 are automated through 129 Dart test cases, all of which pass. The other 22 depend on things a desktop test runner cannot provide: Firebase's servers, the on-device ML model, the camera, Android notifications or a second device. Their manual steps are documented, but their results had not been entered in the SRS sheet when this report was written. The app was demonstrated on a tablet during the final presentation.

Two features were kept deliberately small: the Premium plan uses a simulated payment, and cloud sync merges data instead of replacing it, so an item deleted on one tablet can reappear after the next sync. Both are discussed in Chapter 11.

2 Problem Domain and Background

COURSE OUTCOME CO1

2.1 Domain, stakeholders and motivation

The project belongs to early-years education technology, and more narrowly to handwriting practice on stylus-equipped tablets for children who are just starting school. Four groups have a stake in it.

Table 2.1 Stakeholders and their needs

Stakeholder	Role	What they need from the app
Children, 5 to 8	Primary users. They have no accounts and many cannot yet read well. They practise under a profile created by an adult.	Large controls, pictures, short messages, immediate feedback and a reason to come back.
Parents	Create the account, add children and choose who is practising on the tablet.	Weekly progress and weak areas, challenges with time limits, toy words, reminders, sync and an optional paid plan.
Teachers	Register with a school name, create classes and add students. Often share one tablet between several pupils.	A class overview that marks who needs help, tasks and timed tests, grades with an answer key, written feedback and exportable reports.
Indirect	Schools, the development team, and the service providers (Google Firebase and ML Kit).	An app that is safe for children, inexpensive to run and straightforward to maintain.

There is good evidence that writing letters by hand helps young children learn them. Longcamp and colleagues found that preschool children who practised letters by handwriting later recognised them better than children who practised by typing [7]. James and Engelhardt found that children who printed letters free hand, rather than typing or tracing them, later showed stronger activity in the brain regions used for reading [8]. The second result had the most direct effect on our design: tracing is a useful start, but the child also has to produce the letter without a guide.

2.2 Computing and software engineering concepts applied

Table 2.2 Core concepts and where they appear in KidScribe

Concept	Where it appears	Reason
Data structures	A drawing is held as `List<List<Offset>>` with a parallel `List<List<int>>` of timestamps. Progress is a nested map per child, e.g. `{"A": {"guided": 3, "free": 82}}`. Enumerations such as `StudentStatus` and `AssignKind`.	Plain structures that serialise directly to JSON and can be compared point by point.
Geometric algorithms	Arc-length resampling of strokes; nearest-point tests for coverage and precision; comparison of strokes in order and reversed to detect wrong order or direction (`trace_evaluator.dart`).	Marks a letter on the tablet in a fraction of a second without a server.
Image processing	Variance of the Laplacian over a reduced greyscale copy of a toy photo [9].	Rejects blurred photos before a toy word is saved.
Applied machine learning	ML Kit Digital Ink Recognition for free-hand letters; ML Kit Image Labeling to suggest toy names. Both run on the device.	Recognition without sending a child's writing or photos to a server.

Concept	Where it appears	Reason
Software architecture	BLoC in four layers (screens, blocs and cubits, repositories, data providers); dependency injection through `RepositoryProvider`.	Screens contain no rules, so the rules can be unit-tested off the device.
Object orientation and immutability	Single-purpose classes; immutable `Equatable` state objects with `copyWith`.	The interface rebuilds only when state actually changes.
Storage	A key–value store on the tablet (SharedPreferences) and one Cloud Firestore document per account, protected by security rules.	Offline operation by default, with the cloud as an option.
Data replication	Merge rules during sync: larger number wins, maps are joined key by key, lists are joined by identifier.	Two tablets that disagree never lose a child's progress.
Security	Firebase ID token (JWT) and refresh token; password policy; Firestore rule `request.auth.uid == uid`; an adult check before sensitive screens.	The app never stores passwords and each family can read only its own data.
Testing	Unit tests named after SRS test IDs, with positive and negative cases and isolated fake storage per test.	Every automated test traces back to a line in the SRS.

2.3 Specialist domain knowledge

Handwriting is usually taught in a fixed order. Children start with pre-writing strokes, move to single letters, then to words and finally to sentences. Our five stages follow the same order, and each stage stays locked until the previous one is complete. Stage 1 contains the six strokes that most letters are built from: the standing line, the sleeping line, two slanting lines, a curve and a circle.

How a letter is formed matters as much as how it looks. A letter written bottom-up or with its strokes in the wrong sequence can look correct, but the habit makes writing slow later on. We call the standard formation the *designated manner*: the right number of strokes, in the right order, each running from the green start dot to the orange end dot. KidScribe refuses a letter written any other way, however neat it looks, and tells the child where to start.

Young children's hands are unsteady, so the marking has to be forgiving without becoming meaningless. A point counts as on the shape if it lies within 9% of the canvas width. The tracing score is $100 \times \text{coverage} \times \text{precision}^{0.7}$, which lets a wobbly but complete letter pass while a scribble across the page does not. One, two and three stars are awarded at 55, 70 and 85 points, and the free-hand pass mark is 70%.

Handwriting recognisers work on *digital ink*, which records when each point was drawn as well as where. Our writing pad therefore timestamps every point. It also rejects palm contact: once the stylus has been used, finger touches are ignored, so a hand resting on the glass does not leave marks.

Finally, software for young children carries extra obligations. Children never log in, the app has no advertising, and the sync and payment screens sit behind a year-of-birth check. These choices follow the spirit of the COPPA rule [16] and Google Play's Families policy [15].

3 Literature and Existing Solutions

3.1 Research on handwriting and learning

Longcamp, Zerbato-Poudou and Velay taught preschool children a set of letters either by handwriting or by typing, and found that the handwriting group recognised the letters better in later tests [7]. James and Engelhardt compared printing letters free hand with typing and with tracing; only free-hand printing produced the stronger brain response associated with reading [8]. Read together, the two studies suggest that a handwriting app should make children produce letters themselves rather than only follow a guide. On the technical side we built on ML Kit Digital Ink Recognition [12], which recognises handwriting from timed strokes on the device, and on the variance-of-the-Laplacian sharpness measure described by Pech-Pacheco and colleagues [9].

3.2 Comparable applications

We reviewed two kinds of app that families already use. Dedicated handwriting apps such as LetterSchool and Writing Wizard let a child trace letters over a guide. General early-learning apps such as Khan Academy Kids cover reading and number skills mainly through tap-and-drag activities. Table 3.1 compares what we found typical of each group with KidScribe; individual apps change between versions, so the comparison is by group rather than by product.

Table 3.1 Comparison with existing approaches

Aspect	Tracing apps	General early-learning apps	KidScribe
Writing activity	Trace over a guide with finger or stylus	Mostly tapping and dragging	Trace over numbered points, then write the same letter without a guide
Assessment	Mainly whether the path was followed	Right or wrong answer	Handwriting model and shape comparison blended into a percentage, with a specific tip
Stroke order	Usually demonstrated by animation	Not addressed	Demonstrated by video and numbered dots, and enforced during marking
Beyond letters	Some include words	Reading activities	Words, then sentences with capitals and full stops
Personal content	Some accept a typed word list	None	Toy photographs become practice words; blurred photos are refused
Teacher support	Limited	Some classroom features	Classes, “needs help” flags, tasks, timed tests without help, grades, feedback and reports
Offline use	Usually	Varies	Yes; only sign-up, login, sync and the first model download need a connection

3.3 Where KidScribe fits

We kept the tracing that makes existing apps easy to start with and added a free-hand step that must also be passed, which is the change the research points to. Recognition on its own can only say that a drawing resembles an “A”; the geometric check adds where the letter went wrong and whether it was formed correctly. Toy

words tie practice to objects the child cares about, which none of the apps we looked at did from a photograph. The timed test, which hides every form of help, applies the same idea to assessment: it shows a teacher what a child can write alone.

4 Requirements (SRS Session)

4.1 Writing and tracing the requirements

Requirements were written from the point of view of the three user groups as user stories of the form “As a …, I want …, so that …”. Each story carries a priority and a set of acceptance criteria, at least one of which describes what happens when something goes wrong, such as an empty field or an unclear photo. Every story is then covered by at least two unit tests, one positive and one negative. Requirements, stories and tests therefore form a single chain from FR to US to T, and the final specification contains 24 functional requirements, 33 user stories (14 high, 14 medium and 5 low priority) and 80 unit tests.

Table 4.1 Functional requirements, related stories and main implementation

FR	Requirement	Stories	Priority	Main code
FR_1	Parents and teachers create an account	US_1–3	High	`auth_bloc`, `password_rules`
FR_2	Parents and teachers log in	US_4–5	High	`auth_service`
FR_3	Create and manage child profiles	US_6	High	`children_cubit`, `family_service`
FR_4	Teachers create classes and add students	US_7	High	`classes_cubit`, `family_service`
FR_5	Practise basic strokes	US_8	High	`tracing_cubit`, `trace_evaluator`
FR_6	Practise letters A to Z	US_9	High	`letter_practice_cubit`
FR_7	Check each letter and give instant feedback	US_10	High	`trace_evaluator`, `freehand_checker`
FR_8	Practise common words	US_11	High	`word_practice_cubit`
FR_9	Build sentences and learn basic grammar	US_12–13	Medium	`sentence_cubit`, `sentence_checker`
FR_10	Guide children through five stages in order	US_14	High	`stages_cubit`
FR_11	Letter Match game	US_15	Medium	`match_cubit`
FR_12	Letter Ninja timed game	US_16	Medium	`ninja_cubit`
FR_13	Free Writing on a blank page	US_17	Medium	`free_writing_cubit`
FR_14	Badges, streaks and rewards	US_18	Medium	`badge_service`
FR_15	Daily practice reminders	US_19	Low	`reminder_service`
FR_16	Custom words from a toy photo	US_20	Medium	`custom_word_service`
FR_17	Parents set writing challenges	US_21	Medium	`assign_cubit`, `assignment_service`
FR_18	Teacher overview of class progress	US_22–24	High/Med/Low	`classes_cubit`, `report_service`
FR_19	Progress reports for parents	US_25	High	`progress_cubit`, `progress_service`
FR_20	Save on the device and sync across devices	US_26–27	Med/Low	`progress_service`, `sync_service`
FR_21	Premium features for parents	US_28–29	Low	`premium_service`
FR_22	Written feedback on a child’s work	US_30	Medium	`assignment_service`

FR	Requirement	Stories	Priority	Main code
FR_23	Time limits on tasks and challenges	US_31–32	Medium	`assignment_service`
FR_24	Timed test with no help on screen	US_33	Medium	`my_tasks_cubit`

4.2 Non-functional requirements

The course's SRS template concentrated on functional requirements, so our quality goals were not numbered there. Table 4.2 states them and how each was met.

Table 4.2 Quality goals

Goal	Requirement	How it is met
NFR-1 Offline use	Practice, marking and saving work with no connection.	Data is written to the tablet first; ML Kit runs locally after a one-time model download of about 20 MB. Tests T_61 and T_62 pass.
NFR-2 Responsiveness	Feedback appears immediately after Check is tapped.	All marking runs on the device; a progress indicator covers the short ML step.
NFR-3 Child safety and privacy	No child accounts, no advertising, no payment data, data isolated per family.	Profiles live under an adult account; adult check; per-user Firestore rule; simulated payment only.
NFR-4 Usability for young children	Large targets, little reading, an obvious next step.	Large pill buttons, a single highlighted action per screen, illustrations instead of icons, short messages, palm rejection.
NFR-5 Maintainability	Business rules testable without a device.	Layered BLoC design with downward-only imports; no analyzer warnings; 129 unit test cases.
NFR-6 Platform	Android tablets with a stylus.	Android build; a finger also works when no stylus is available.

4.3 Context and sustainable development

Several requirements come from the setting the app is meant for. In many classrooms one tablet is shared between several children, so a teacher can choose which pupil is writing and the work is stored under that pupil rather than under the teacher's account. Internet access can be slow or expensive, which is why practice works offline and the handwriting model runs on the device; a family needs data only to sign up, to sync and to download the model once. There is no server of our own to keep running, since Firebase handles login and the optional sync, so the cost of operating the app stays close to zero for a school or a small team. The demonstration price is shown in taka (৳199 a month) and every learning feature is free, so cost does not stand between a child and the core purpose of the app. In this way the project contributes, on a small scale, to the United Nations Sustainable Development Goal on quality education [18].

4.4 Changes after the SRS review

The SRS was revised several times between the SRS session and the end of the project. Table 4.3 lists the changes and the reasoning behind each.

Table 4.3 Revisions to the SRS

Area	First version	Final version	Reason
Assessment	21 FRs, 29 stories and 68 tests, all about practice.	FR_22 to FR_24 added with US_30 to US_33 and T_69 to T_80: written feedback, deadlines, and a timed test with no help on screen.	Practice alone gave a teacher no way to see what a child can write unaided, or to set a deadline.
Who logs in	"Parents and teachers create an account", with children's access left open.	Children never log in. They practise under a profile, and an adult chooses the active learner first.	Young children cannot manage passwords, and child data stays under an adult's control.
Payment	"The premium plan can be bought."	A clearly labelled simulated payment with no card details, plus a way to simulate a failed payment (T_66).	Real payments were outside our scope, and handling card data carries legal duties we could not meet.
Test execution	A single status column.	An added column stating whether each test is automated or run by hand on the emulator.	Some tests cannot run without a device, and the sheet now says so.
Sync	"The same progress shows on all devices."	A note that sync merges and never deletes, so removed items can return.	We chose never to lose progress and documented the side effect.
Status	9 stories done, 2 in progress, 18 to do; 26 tests passing.	All 33 stories done; 58 tests passing automatically.	The SRS sheet also served as our backlog (Section 9.2).

5 System Design (Design Session)

COURSE OUTCOME CO3

5.1 Architecture

KidScribe is a single Android application with no server of its own. Internally it follows the BLoC architecture in four layers (Figure 5.1). Events and calls travel down; states and results travel back up. A screen never calls a service directly, and a service has no knowledge of any screen. `main.dart` assembles the layers: a `MultiRepositoryProvider` supplies the services, a `MultiBlocProvider` supplies the application-wide `AuthBloc`, and each screen creates its own cubit, which is disposed of when the screen closes.

Presentation `lib/screens, lib/widgets`	Draws only. `BlocBuilder` redraws on a new state; `BlocListener` handles one-off events such as the praise card. 30 screen files and 7 shared widget files.
↓ events and method calls	↑ immutable states
Business logic `lib/blocs`	All rules and state: one Bloc (`AuthBloc`, driven by named events) and 26 Cubits, roughly one per screen. States are immutable and `Equatable`.
↓ method calls	↑ models and results
Repositories `lib/services`	Progress, Family, Assignment, Badge, Sync, Report, CustomWord, Premium, Reminder and Auth services, together with the pure checkers TraceEvaluator, FreehandChecker and SentenceChecker.
↓ JSON, ink, images	↑ maps, documents, labels
Data providers `external packages`	SharedPreferences, Firebase Authentication, Cloud Firestore, ML Kit Digital Ink Recognition, ML Kit Image Labeling, Android notifications and the app's documents folder.

Figure 5.1 The four layers of the application and what passes between them.

The learning path is fixed (Figure 5.2); a stage opens only after the one before it is finished.

Stage 1	Stage 2	Stage 3	Stage 4	Stage 5
Strokes Six pre-writing shapes	**Letters** A–Z, traced then written free hand	**Words** 25 built-in words plus toy words	**Sentences** 13 sentences with capitals and full stops	**Grown-up zone** Sync and Premium behind an adult check

Figure 5.2 The five learning stages.

5.2 Components and data model

The main data classes all provide `toMap` and `fromMap`. `ChildProfile` holds an identifier, a first name and an age, and `SchoolClass` a name and a list of students. `Assignment` is the largest model: it records the title, whether it is a practice task or a test, who set it, its items (strokes, letters, words or sentences), whether they must be done in order, the deadline, the test length and start time, the steps completed, and after grading the mark, answer key and note. `FeedbackNote` attaches a comment either to a whole profile or to one letter. The checkers return `TraceResult` (coverage, precision, score, stars and a tip) and `FreehandResult` (percentage, pass flag, weakest zone, tip and a one-line summary for adults).

Every stored key ends with a *scope* that identifies the child the data belongs to. The scope is chosen in a fixed order: the pupil a teacher has selected on a shared tablet, otherwise the parent's active child, otherwise the logged-in account. This single rule lets several children use one tablet without their progress mixing.

Table 5.1 Stored data

Key or document	Example	Owner
`trace_progress_<scope>`	`{"s1_circle": 3}` (stars per stroke)	ProgressService
`letters_v2_<scope>`	`{"A": {"guided": 3, "free": 82}}`	ProgressService
`words_v1_<scope>`	`{"CAT": 1}`	ProgressService
`practice_log_<scope>`	`{"2026-07-15": 12}` (checks per day)	ProgressService, BadgeService
`assignments_<scope>`, `feedback_<scope>`	Lists of tasks, tests and notes	AssignmentService
`custom_words_<scope>`	Toy words with photo paths	CustomWordService
Firestore `kidscribe_sync/{uid}`	`{"data": {…}, "updatedAt": …}`, each value tagged with its type	SyncService

5.3 Alternatives and trade-offs

Table 5.2 Design decisions and the alternatives considered

Decision	Chosen	Considered	Rationale and cost
State management	flutter_bloc, with Cubits and one Bloc for login	`setState`, Provider, Riverpod	Keeps rules out of widgets and testable. Costs more files; a full Bloc is used only for login, where named events make the flow clearer.
Handwriting assessment	On-device ML Kit (55%) blended with our shape check (45%)	Shape check alone; cloud vision API; a model of our own	Shape alone can be gamed and ML alone cannot say what to fix. A cloud API needs a connection and would send children's writing away. We had no data to train a model.
Local storage	SharedPreferences with JSON strings	sqflite, Hive	Data is small and read per child without queries. A whole map is rewritten on each save, which would not scale to large data.
Accounts	Firebase Authentication	Our own server with hashed passwords and JWTs	Nothing to host or secure, and tokens refresh automatically. Login needs a connection, and Firebase has no role field, so we store the role ourselves.
Synchronisation	One Firestore document per account, merged on both sides	Last write wins; no sync	Progress is never lost when two tablets disagree. Deletions do not propagate.
Pen input	Points held in a local controller and handed over on Check	Every point passed through the cubit	The stylus reports about 60 points a second; routing each through a bloc would slow drawing for no benefit.
Platform	Flutter, Android build	Kotlin, React Native	`CustomPainter` gives a smooth canvas and the same code could later target iPad. Only Android is configured and tested.

5.4 Design review and resulting changes

The questions in Table 5.3 were raised while the design was reviewed and as we began to build on it. Each one led to a visible change.

Table 5.3 Design questions and changes

Question	Issue	Change made
If each screen has a cubit, should pen input go through it?	Sixty state updates a second while writing would make the pen lag.	A `WritingPadController` holds short-lived pen state; the cubit receives finished strokes only on Check. Recorded in `ARCHITECTURE.md`.
How is a result shown exactly once?	A listener can fire again on rebuild and repeat a pop-up.	States carry counters such as `resultTick` and `freeTick`; listeners react only when the counter changes.
Whose progress is saved on a shared classroom tablet?	Saving under the logged-in teacher mixes every pupil together.	The scope rule in Section 5.2.
Where is the user's role kept?	Firebase Authentication stores email, password and name, not Parent or Teacher.	The role is stored on the device and copied to Firestore when sync is enabled.
Should a correct-looking letter pass if formed the wrong way?	It would reinforce habits that slow writing later.	An order-and-direction check (`OrderCheck`) that both tracing and free-hand marking must pass.

6 Implementation (Implementation Session)

COURSE OUTCOMES CO1 AND CO3

6.1 Repository structure

The repository follows the department's layout, with the Flutter project in `/code` and the documents in `/doc`. The line counts refer to the Dart files under `code/lib`.

```
kidscribe-spl2-project/
├── code/                          Flutter project (Android)
│   ├── lib/
│   │   ├── main.dart              wires services, AuthBloc and the app
│   │   ├── firebase_options.dart  Firebase settings (maintained by hand)
│   │   ├── screens/    30 files   8,977 lines   user interface
│   │   ├── widgets/     7 files   2,501 lines   writing pad, buttons, mascot, artwork, adult
check
│   │   ├── blocs/      29 files   3,459 lines   1 Bloc and 26 Cubits
│   │   ├── services/   15 files   2,382 lines   storage, Firebase, ML Kit, checkers
│   │   ├── models/      4 files     303 lines   data classes
│   │   ├── data/        3 files     357 lines   stroke and letter templates, words, sentences
│   │   └── theme/       2 files     161 lines   colours and theme
│   ├── unit_testing/
│   │   ├── test/       16 files   1,920 lines   129 test cases
│   │   ├── results/flutter_test_output.txt
│   │   └── KidScribe_Unit_Testing.pdf
│   ├── assets/art/                Fluent Emoji illustrations and licence
│   ├── assets/sounds/             success sound
│   ├── android/                   Gradle build, package com.example.kidsscibe
│   ├── ARCHITECTURE.md
│   └── pubspec.yaml
├── doc/
│   ├── srs.pdf
│   ├── design.pdf
│   ├── final-presentation.pdf
│   └── final-report.pdf
└── README.md                      build, run and test instructions
```

6.2 Communication between components

Table 6.1 Messages and data at each boundary

Boundary	What is passed	Effect
Screen to Bloc	An event object: `context.read<AuthBloc>().add(AuthLoginRequested(email: e, password: p))`	`AuthBloc` emits a submitting state, then authenticated or an error.
Screen to Cubit	A method call: `cubit.checkFree(strokes, times, side)`	The cubit marks, saves and emits a new state.
Cubit to Screen	An immutable state, e.g. `LetterPracticeState` with `freeBest`, `freeResult`, `freeTick`, `newBadges`	`BlocBuilder` redraws; `BlocListener` shows the praise card once.
Service and SharedPreferences	JSON strings under scoped keys such as `letters_v2_<scope>`	Progress survives restarts.
Service and Firestore	`kidscribe_sync/{uid}` with typed values, e.g. `{"t": "s", "v": "…"}`	Local and remote data are merged and written back to both.

Boundary	What is passed	Effect
App and Firebase Auth	Email and password once, then an ID token (JWT) and refresh token held by Firebase	The password is never stored by the app.
Service and ML Kit	An `Ink` object of timed points, or a photo path	Ranked letter candidates, or labels such as “Teddy bear”.

6.3 End-to-end example: a free-hand letter A

Figure 6.1 follows a single free-hand attempt from the moment the child taps Check. The writing pad has already recorded the strokes and a timestamp for every point. The cubit first marks itself busy, which shows a progress indicator, and passes the drawing to `FreehandChecker`. The checker asks ML Kit what the drawing looks like and converts the rank of “A” among the candidates into the machine-learning part of the score. It then scales the drawing onto the model letter, measures coverage separately in the top, middle and bottom thirds so that the weakest third can be named, blends the two parts in a 55 to 45 ratio, and finally runs the order check; a letter drawn in the wrong order or direction cannot pass whatever its score. The cubit saves the result if it beats the child’s best, asks `BadgeService` for any newly earned badges and emits one new state. Because the state’s `freeTick` has increased, the screen shows the praise card exactly once, and the next letter opens if the traced half has also been passed and the free-hand score is at least 70%.

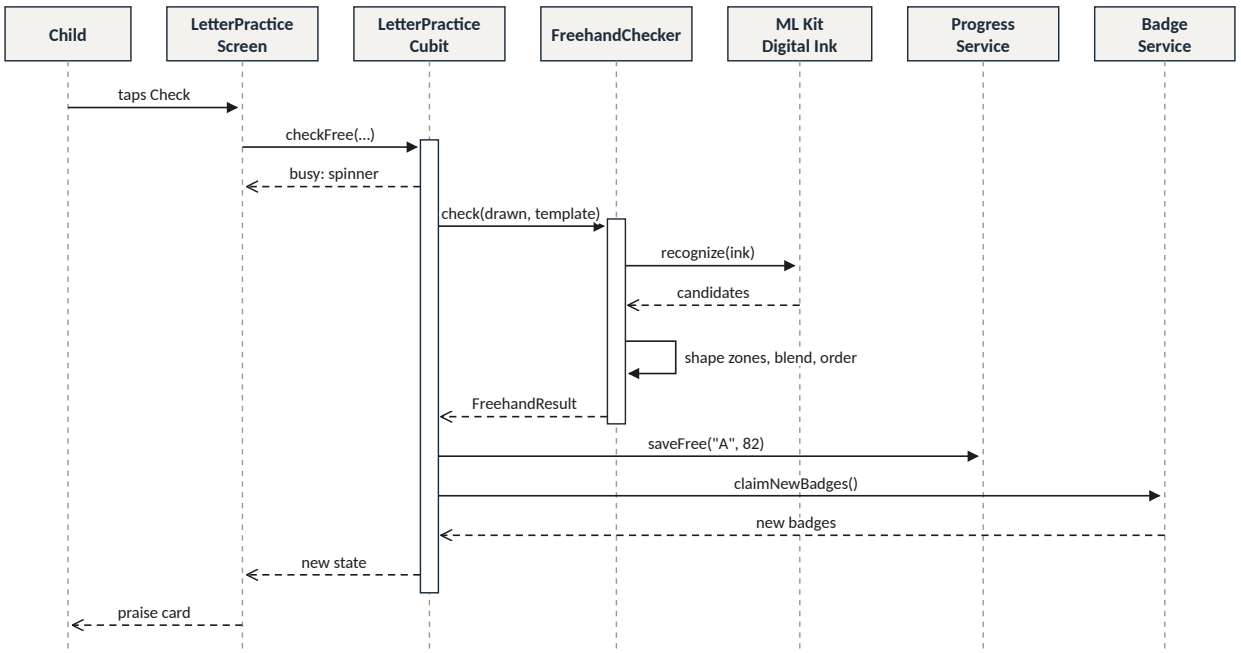


Figure 6.1 Sequence of calls when a free-hand letter is checked. Solid arrows are calls, dashed arrows are returns or new states.

6.4 Classes, methods and code size

We measured `code/lib` with the tree-sitter Dart grammar, which works on the syntax tree rather than on text patterns. Enumerations, mixins and extensions are counted as classes; constructors are not counted as methods. “Logic” refers to `blocs`, `services`, `models` and `data`, and “UI” to `screens`, `widgets` and `theme`.

Table 6.2 Size and structure of the code base (lengths in lines)

Measure	Value	Measure	Count	Mean	Max
Dart files	92	Classes	233	68.6	719
Physical lines	18,255	Methods	736	17.8	262
Blank lines	1,381	logic	403	9.1	163
Comment lines	937	UI	332	28.5	262
Code lines	15,937	Median method			8 lines
Test files / lines / cases	16 / 1,920 / 129	Methods of 20 lines or fewer			76%

The figures on slide 8 of our presentation (240 classes and 759 methods) were taken earlier with the Dart analyzer on a local copy; the difference comes from files that changed before the code was pushed. The pattern is unchanged. Logic methods are short, averaging nine lines, while the long methods are almost all in the interface. A Flutter app naturally has many classes, since every widget, state object, model and service is one, which is why the class count is high while most classes stay small.

Table 6.3 Longest methods and largest class

Method or class	Lines	Reason and planned action
`_PennyPainter.paint` (mascot)	262	Draws the pencil mascot shape by shape. It contains no logic, so it was left as one sequence of drawing calls.
`AssignScreen.build`	184	The form for creating a task or test. It should be divided into smaller widgets.
`DemoVideo.build`	168	The animation that demonstrates how a letter is written.
`FreehandChecker.check`	163	The longest logic method. Its four stages (recognition, shape, order, feedback) should become separate private methods.
`ProgressScreen._body`	155	Lays out the weekly chart, completed items and weak areas.
Class `_HomeView`	719	The home screen shows different cards for parents, teachers and children; the cards could move into their own files.

6.5 Changes in the final phase

Once the features were working we spent the last part of the project on the app's appearance, on tidying the code and on testing. Table 6.4 compares the earlier build with the version in the repository.

Table 6.4 Changes made after the main implementation

Area	Before	After
Illustrations	System emoji inside text and buttons, which look different on each manufacturer's tablets and appear as empty boxes on some older devices.	A `KidArt` widget that draws bundled Fluent Emoji images (MIT licence). They look the same everywhere, and locked items appear as a grey silhouette.
Messages	Cubit messages mixed text and emoji.	Messages are plain text and pictures come only from the artwork, which simplifies testing and later translation.
Visual design	The default Material appearance.	A violet brand colour with a single orange action per screen, pill buttons that sink when pressed, rounded cards, a dark home screen, the Penny mascot and short entrance animations. All colours are defined in `app_colors.dart`.

Area	Before	After
Static analysis	`flutter analyze` also scanned generated build and Android files.	`analysis_options.yaml` excludes `build/` and `android/`; our own code reports no issues.
Assessment	No deadlines, tests or written feedback.	FR_22 to FR_24: deadlines that close a task automatically, extra time on request, timed tests without help, grades with an answer key, and feedback per profile or per letter.
Tests	A handful of manual checks.	A `unit_testing` folder with 16 test files, test names that begin with the SRS test ID, isolated fake storage for each test and the saved output of the last run.

7 Tools, Version Control, Testing and CI/CD

COURSE OUTCOME CO4

7.1 Tools and alternatives

Table 7.1 Main tools and libraries (versions from `pubspec.lock`)

Purpose	Chosen	Alternative	Reason and known limits
Language and UI	Dart 3.11, Flutter 3.38	Kotlin with Jetpack Compose	Hot reload while designing screens and a smooth custom canvas. Android only at present.
State	flutter_bloc 8.1.6, equatable 2.1.0	Provider, Riverpod	Clear layering and simple unit tests, at the cost of more files.
Login	firebase_auth 5.7.0	Own server	Nothing to host; a connection is needed to log in.
Sync	cloud_firestore 5.6.12	Realtime Database	Suits one JSON document per account; a document is limited to 1 MiB.
Local storage	shared_preferences 2.5.5	sqflite, Hive	Simple and adequate for the data size.
Handwriting	google_mlkit_digital_ink_recognition 0.14.2	Cloud OCR	Offline after one download; good with single letters, weaker with untidy whole words.
Toy names	google_mlkit_image_labeling 0.14.2, image 4.9.2	Cloud vision	Runs locally; labels are suggestions and the parent chooses the word.
Reminders	flutter_local_notifications 17.2.4, timezone 0.9.4	Firebase Cloud Messaging	No server required; Android 13 and later ask the user for permission.
Code quality	flutter_lints 4.0.0	Custom rules	Standard Flutter rule set; no reported issues.
Environment	Android Studio emulator, VS Code, physical tablets	–	ML Kit and the camera need a real or emulated Android device.

7.2 Version control

The code is hosted on GitHub at github.com/Kabirohi/kidscribe-spl2-project on the `main` branch. The history contains 22 commits from all three members. Each commit adds one feature and was made by the member who built it, for example “Add Stage 4: building sentences” and “Add tasks, timed tests, feedback and reminders”.

The repository has a clear weakness. For most of the term each of us worked on a local copy, and the GitHub repository was created late, so every commit is dated 27 or 28 October 2026. The history reads well feature by feature but does not show how the work progressed week by week, and no branches or pull requests were used. Section 11.4 describes how we would handle this next time.

Table 7.2 Commits by member

Member	Commits	Content, in order
Zuhaaer Mustafa Sadab	6	Android project set-up; theme and colour palette; artwork and sounds; shared screen components and the writing pad; login and sign-up with Firebase; completed artwork and shared widgets.

Member	Commits	Content, in order
Md Shahib Al Kabir	11	Stroke, letter and word templates (several uploads); stages opening in order; Stages 1 and 2; Stage 3; Stage 4; the unit tests.
Hasan Mahi	5	The three games; badges and streak; child profiles, classes and home screen; tasks, timed tests, feedback and reminders; toy words, Premium, sync, reports and documentation.

7.3 Testing

Because the rules live in cubits and services rather than in screens, they can be tested directly with `flutter_test` without a device. Each test name begins with the SRS test ID it covers, for example `T_23 a traced stroke is accepted`. A shared helper gives every test an empty fake store, so tests cannot see each other's data, and can add a slight wobble to a perfect template to imitate a child's hand. Some SRS tests needed several checks, which is why 58 SRS tests became 129 test cases. The suite runs with `flutter test unit_testing/test` in about 23 seconds; the last line of our run reads `00:23 +129: All tests passed!`

Table 7.3 Automated test suite

File	Scope	Cases	SRS tests
`password_rules_test`	Password policy at sign-up	7	T_7–T_11
`family_service_test`	Storing profiles and classes	9	T_17, T_19–T_21
`profile_cubits_test`	Validation when adding a child or class	6	T_18, T_22
`trace_evaluator_test`	Marking traced strokes and letters	10	T_23–T_28
`sentence_checker_test`	Stage 4 grammar rules	11	T_31–T_34
`stages_and_progress_test`	Stage locking, progress, persistence	10	T_35, T_36, T_59–T_62
`letter_match_test`	Letter Match	6	T_37, T_38
`badges_test`	Badges and streak	14	T_43, T_44
`toy_words_test`	Blur check and word rules	8	T_48
`assignment_service_test`	Tasks, deadlines, tests, grades	16	T_49, T_50, T_55, T_72–T_76, T_78–T_80
`teacher_dashboard_test`	Class overview, tasks without a goal	10	T_51–T_54, T_56
`report_service_test`	Creating and saving reports	4	T_57, T_58
`premium_test`	Simulated payment and locked report	7	T_65–T_68
`feedback_test`	Written feedback	5	T_69–T_71
`my_tasks_test`	Child's task list and test mode	6	T_77
Total		**129**	**58 SRS tests, all passing**

The remaining 22 SRS tests rely on something outside our code and cannot run under `flutter test` (Table 7.4). Their steps are written out in `KidScribe_Unit_Testing.pdf`. They are still marked "Not Run" in the SRS sheet because their results had not been formally recorded when this report was written. The live demonstration covered sign-in, the free-hand check and both timed games, but we did not treat it as a test run.

Table 7.4 Tests that require a device

Tests	Dependency
T_1–T_6, T_12–T_16	Sign-up and login use Firebase's servers.
T_29, T_30, T_39–T_42	Handwriting recognition uses the ML model on an Android device.
T_45, T_46	Reminders are delivered by Android.
T_47	Adding a toy word uses the camera.
T_63, T_64	Sync needs two devices and a connection.

7.4 Planned CI/CD pipeline

The repository does not yet contain a pipeline. The workflow below is the one we intend to add as `.github/workflows/ci.yml`. On each push or pull request it installs Flutter, runs the analyzer and the 129 unit tests, builds a release APK and attaches it to the run. The device-dependent tests would remain manual until integration tests are added (Section 11.3).

```
name: KidScribe CI
on:
  push:          { branches: [ main ] }
  pull_request:  { branches: [ main ] }

jobs:
  test-and-build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: code
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - run: flutter pub get
      - run: flutter analyze
      - run: flutter test unit_testing/test
      - run: flutter build apk --release
      - uses: actions/upload-artifact@v4
        with:
          name: kidscribe-apk
          path: code/build/app/outputs/flutter-apk/app-release.apk
```

8 Professional, Ethical and Legal Considerations

COURSE OUTCOME CO4

8.1 Licences of third-party packages

All packages are taken from `pubspec.yaml` with exact versions from `pubspec.lock`. Each is published under a permissive open-source licence that allows academic and commercial use provided the notices are retained, and Flutter displays these notices on its built-in licences page.

Table 8.1 Dependency licences

Package	Version	Licence	Purpose
Flutter SDK, Dart SDK	3.38 / 3.11	BSD-3-Clause	Framework and language
flutter_bloc, bloc	8.1.6, 8.1.4	MIT	State management
equatable	2.1.0	MIT	Value equality for states
firebase_core, firebase_auth, cloud_firestore	3.15.2, 5.7.0, 5.6.12	BSD-3-Clause	Login and sync
google_mlkit_digital_ink_recognition, google_mlkit_image_labeling	0.14.2	MIT	Plugins for handwriting and labelling; the models themselves fall under Google's ML Kit terms
shared_preferences, path_provider, image_picker	2.5.5, 2.1.5, 1.2.3	BSD-3-Clause	Storage, folders, camera
flutter_local_notifications	17.2.4	BSD-3-Clause	Reminders
timezone	0.9.4	BSD-2-Clause	Local-time scheduling
image	4.9.2	MIT	Blur detection
google_fonts	6.3.3	Apache-2.0	Font loading
confetti, audioplayers	0.8.0, 6.7.1	MIT	Celebration effect and sound
cupertino_icons	1.0.x	MIT	Icons
flutter_lints (development)	4.0.0	BSD-3-Clause	Lint rules

8.2 Artwork, fonts and sound

The illustrations in `assets/art` come from Microsoft's Fluent Emoji collection, released under the MIT licence [14], and the licence file is kept in the same folder. The interface fonts, Fredoka and Nunito, are loaded through Google Fonts and licensed under the SIL Open Font Licence 1.1. The Penny mascot was drawn in code by the team and uses no external image. The success sound in `assets/sounds` does not yet have its origin recorded in the repository; before any public release we will document its source and licence or replace it with a sound of our own.

8.3 Privacy, safety and ethics

Children never create accounts. Only an adult signs up, and a child profile contains just a first name and an age. All practice data, tasks and toy photos are kept on the tablet by default. Sync is switched on by a parent from behind the adult check, and the Firestore rule allows each account to read and write only its own docu-

ment. Both machine-learning models run locally, so neither a child's handwriting nor the toy photos are sent to a server for analysis.

The app does not keep passwords; Firebase verifies them and returns tokens. The `UserModel` class still contains a `password` field from an early version. It is always empty and will be removed. The app includes no advertising or analytics packages. The payment screen states plainly that it is a demonstration, never asks for card details and takes no money.

We also paid attention to tone. Feedback tells the child what to try next and never suggests that they are bad at writing, and a missed day resets the streak quietly. The adult check before sync and payment asks for a year of birth, the same light barrier many children's apps use; it stops a curious child but would not stop a determined one.

8.4 Professional responsibility

We have tried to report the project as it stands. Tests that have not been formally run are marked as such rather than as passing. The README warns that running `flutterfire configure` would overwrite the hand-maintained Firebase settings and change the Android package name, and it explains the sync limitation in plain language. Where we used open-source work, its licence travels with it.

9 Project Management and Feasibility

COURSE OUTCOME CO6

9.1 Milestones

Table 9.1 Planned milestones and what was delivered

No.	Milestone	Planned output	Delivered and evidence
M1	SRS session	User stories, acceptance criteria and test plan	21 FRs, 29 stories, 68 tests; accounts, login, strokes, letters and feedback already working (9 stories done, 26 tests passing)
M2	Design session	Architecture, data model and main flows	Four-layer BLoC design, scope rule, storage keys and merge-based sync; written up in `ARCHITECTURE.md` and the design document
M3	Learning stages	Stages 1 to 4 with marking	Strokes, letters with tracing and ML free-hand check, words and sentences
M4	Games and motivation	Three games, badges and streak	Letter Match, Letter Ninja and Free Writing; badges derived from recorded progress
M5	Adult features	Profiles, classes, reports, reminders, toy words, sync, Premium	All implemented, with Premium simulated and sync merge-only
M6	SRS update	Assessment features	FR_22 to FR_24 added and implemented
M7	Unit testing	Automated tests for every SRS test that can run off-device	Unit testing document dated 21 October 2026; 129 of 129 cases pass
M8	Completion	Redesign, repository, presentation, report	New artwork and visual design; code pushed on 27 and 28 October; final presentation with live demo; this report

9.2 Tracking the work

The SRS sheet served as our backlog. Each user story has a status (To Do, In Progress or Done) and each test has a result, so the sheet showed at any point what was left. At the SRS session it listed 9 stories as done, 2 in progress and 18 still to do. We took the high-priority stories first, so that accounts, strokes, letters, feedback and the stage sequence were working before we moved on to games and adult features. Work was divided by feature area (Chapter 10); we agreed on the shared components before building on them and updated the sheet as stories were completed. We did not use a separate board or issue tracker, which sometimes made it hard to see who was waiting on whom.

9.3 Technical problems and their resolution

Table 9.2 Main technical problems

Problem	Cause	Resolution	Files and owner
Pen lag and stray marks	About 60 points a second passed through state management; a resting palm registered as touches.	Pen state kept in `WritingPadController` and handed over on Check. Finger input ignored once the stylus has been used.	`writing_pad.dart`; Zuhaaer

Problem	Cause	Resolution	Files and owner
Wrongly formed letters passing	Coverage and precision ignore stroke order and direction.	Strokes resampled to equal length and compared in order; a match only when reversed or at another step number fails with a specific tip.	`trace_evaluator.dart`, `freehand_checker.dart`; Kabir
Scores without guidance	ML Kit reports which letter a drawing resembles, not which part is weak.	Drawing scaled onto the model letter and coverage measured in thirds; the weakest third drives the tip; 55/45 blend.	`freehand_checker.dart`; Kabir
Broken build and missing role	`flutterfire configure` rewrote the Firebase settings and package name; Firebase Auth has no role field.	Settings file maintained by hand with a warning in the README; role stored locally and copied to Firestore on sync.	`firebase_options.dart`, `auth_service.dart`; Zuhaaer
Conflicting data after sync	The last tablet to sync overwrote the other's progress.	Merge instead of overwrite: larger numbers win, maps and lists are joined. The deletion side effect is documented.	`sync_service.dart`; Hasan
Tests failing off-device	Storage, Firebase and ML Kit are unavailable to the desktop test runner.	Fake storage per test, direct tests of the pure checkers, and device-dependent tests separated with written steps.	`helpers.dart`; Kabir

9.4 Feasibility and cost

The project needed no purchases. We developed on our own laptops, tested on Android tablets and phones we already had and on the Android Studio emulator, and a stylus, while helpful, is not required. Firebase's free Spark plan covers authentication and Firestore at our scale, the ML Kit on-device models are free, and there is no server to rent, so the running cost during the project was nil. Publishing on Google Play would need a one-time developer registration fee of US$25, and a real Premium plan would go through Google Play Billing, which retains a service fee on each sale. On schedule, we did not drop any requirement. Instead we limited the depth of two features, the simulated payment and merge-only sync, so that every story could be completed and tested on time.

10 Teamwork and Individual Contributions

COURSE OUTCOME CO6

10.1 Responsibilities

Table 10.1 Individual contributions

Member	Main responsibilities	Commits	Dart lines added
Hasan Mahi 230042129	Letter Match, Letter Ninja and Free Writing; badges and streak; child profiles, classes and home screen; tasks, deadlines, timed tests, grading and feedback; reminders; toy words and blur check; Premium; Firestore sync; progress reports; README.	5	8,586
Zuhaaer Mustafa Sadab 230042132	Android project set-up; theme and palette; visual design and redesign; artwork, sound and the Penny mascot; shared widgets and animations; writing pad with palm rejection; Firebase login and sign-up; password rules; adult check.	6	3,983 and 127 art files
Md Shahib Al Kabir 230042157	Stroke, letter and word templates; the five stages and their locking; Stages 1 and 2 with TraceEvaluator and FreehandChecker; ProgressService; Stage 3 words; Stage 4 sentences with SentenceChecker; the 129 unit tests and the testing document.	11	7,724 incl. 1,920 test lines

Line counts are taken from the Git history for `.dart` files and are only a rough indicator. Design, artwork, device testing, the SRS and the presentation do not appear in them.

10.2 Coordination between modules

Working in parallel depended on a few shared components that were agreed early and then kept stable. Zuhaaer's `WritingPad` is used by Kabir's stages and by Hasan's games and timed tests, so palm rejection and timestamps behave identically everywhere. Kabir's `ProgressService`, with its scope rule, is the single source that Hasan's badges, class overview, reports and sync all read from. Zuhaaer's `AuthService` and `AuthBloc` determine who is logged in and with what role, which Hasan's profile and class screens build on. Because all screens use the same `KidButton`, `KidCard`, `KidHeader` and `KidArt` components, the redesign at the end touched only a handful of files.

10.3 Handling disagreements

The longest discussion we had was about whether a letter that looks right but was written in the wrong stroke order should pass. One view was that young children lose heart if a neat letter is rejected. The other was that good habits are the purpose of the app, and that the wrong order makes writing slow later. We tried both versions on the tablet before deciding. The strict rule stayed, but we rewrote its message to be specific and encouraging ("Start at the green dot") and made sure the demonstration video and numbered dots show the correct order before the child tries again. Most other disagreements concerned scope, and those we settled by returning to the priorities in the SRS.

11 Results, Limitations and Future Work

COURSE OUTCOME CO6

11.1 Status of each requirement

Table 11.1 Requirement status and verification

FR	Requirement	Status	Verification
FR_1	Create an account	Done	T_7–T_11 automated, passing; T_1–T_6 manual (Firebase)
FR_2	Log in	Done	T_12–T_16 manual (Firebase)
FR_3	Child profiles	Done	T_17–T_19 automated, passing
FR_4	Classes and students	Done	T_20–T_22 automated, passing
FR_5	Basic strokes	Done	T_23, T_24 automated, passing
FR_6	Letters A to Z	Done	T_25, T_26 automated, passing
FR_7	Instant letter feedback	Done	T_27, T_28 automated, passing
FR_8	Common words	Done	T_29, T_30 manual (ML model)
FR_9	Sentences and grammar	Done	T_31–T_34 automated, passing
FR_10	Five stages in order	Done	T_35, T_36 automated, passing
FR_11	Letter Match	Done	T_37, T_38 automated, passing
FR_12	Letter Ninja	Done	T_39, T_40 manual (ML model)
FR_13	Free Writing	Done	T_41, T_42 manual (ML model)
FR_14	Badges and streaks	Done	T_43, T_44 automated, passing
FR_15	Daily reminders	Done	T_45, T_46 manual (Android)
FR_16	Toy camera words	Done	T_48 automated, passing; T_47 manual (camera)
FR_17	Parent challenges	Done	T_49–T_51 automated, passing
FR_18	Teacher class overview	Done	T_52–T_58 automated, passing
FR_19	Progress reports	Done	T_59, T_60 automated, passing
FR_20	Offline storage and sync	Done	T_61, T_62 automated, passing; T_63, T_64 manual (two devices)
FR_21	Premium plan	Done (simulated)	T_65–T_68 automated, passing; payment is simulated
FR_22	Written feedback	Done	T_69–T_71 automated, passing
FR_23	Time limits	Done	T_72–T_76 automated, passing
FR_24	Timed test without help	Done	T_77–T_80 automated, passing

11.2 Known limitations

The app runs only on Android; there is no iPad or web version. Twenty-two of the 80 SRS tests have written procedures but no recorded result yet. Sync never deletes, so a child or task removed on one tablet returns after the next sync, and because each account's data sits in a single Firestore document, a very large class could eventually approach the 1 MiB document limit. Premium is not connected to a real payment provider.

The content is also limited. Stage 2 covers only capital letters, and there are 25 built-in words and 13 sentences besides any toy words. The first free-hand check needs a connection to download the handwriting model of about 20 MB. Reports are plain text files, and the adult check asks only for a year of birth. Finally, we have tested that the app works, not how much it improves children's handwriting; answering that would need a proper study with teachers and the consent of parents.

11.3 Future work

The most useful next step for learners would be lowercase letters and Bangla script, since many of the children we have in mind learn both. On the technical side, sync should record deletions as small markers so that they reach other tablets, and data should be split into one document per child. The 22 manual tests can be automated with Flutter's `integration_test` package on an emulator and added to the pipeline in Section 7.4. Connecting Google Play Billing would make Premium real, since the locking logic is already in place. Teachers would benefit from PDF reports with the weekly chart and from a web page for viewing classes on a laptop. In the code, `FreehandChecker.check` and `_HomeView` should be divided into smaller parts and the unused `password` field removed from `UserModel`. With a school's permission, we would also like to run a small trial comparing children's writing before and after a few weeks of use.

11.4 Reflection

The clearest lesson concerns version control. Setting up GitHub late saved no effort and cost us a history that shows how the project developed. Next time the repository would exist from the first week, with feature branches and pull requests so that progress and review are visible to everyone, including the course teachers.

The architecture paid off most during testing. Because the BLoC layers kept rules out of the screens, most tests were short and quick to write, and the parts that were difficult to test were exactly those that talk to the outside world. We also learned that not every piece of state belongs in the state manager: fast, short-lived data such as pen points works better kept locally, with only the result handed to the cubit. Lastly, we found it better to write limitations down, such as the simulated payment and merge-only sync, than to leave users to discover them.

12 Glossary and References

12.1 Glossary

Table 12.1 Terms used in this report

Term	Meaning
APK	Android Package, the file used to install an app on Android.
BLoC	Business Logic Component; an architecture in which screens send events, a bloc applies the rules and the screen redraws from the resulting state.
Cubit	A simpler form of bloc driven by method calls instead of event objects.
Coverage, precision	The share of the target shape covered by the pen, and the share of the ink that stayed on the shape.
Designated manner	The standard way of forming a letter: correct number, order and direction of strokes.

Term	Meaning
Digital ink	Handwriting stored as strokes of points, each with coordinates and a time.
Equatable	A Dart package that compares objects by value, so the interface rebuilds only on real change.
FR, US, T	Functional requirement, user story and unit test, as numbered in the SRS.
Firestore	Google's cloud document database, used here only for optional sync.
JWT	JSON Web Token; the signed ID token issued by Firebase after login.
ML Kit	Google's machine-learning library for mobile apps, running on the device.
Scope	The child, or account, to which a piece of stored data belongs.
SharedPreferences	A simple key–value store on the device.
Variance of the Laplacian	A measure of edge sharpness in an image; a low value indicates blur.

12.2 References

1. Group 18 (H. Mahi, Z. M. Sadab, M. S. A. Kabir), *KidScribe Software Requirements Specification*, updated sheet, IUT CSE, Oct. 2026. Available as `/doc/srs.pdf`.
2. Group 18, *KidScribe Design Document*, IUT CSE, 2026. Available as `/doc/design.pdf`.
3. Group 18, *KidScribe Unit Testing*, IUT CSE, 21 Oct. 2026. In `code/unit_testing/`.
4. Group 18, *KidScribe Final Presentation*, IUT CSE, Oct. 2026. Available as `/doc/final-presentation.pdf`.
5. KidScribe source repository, github.com/Kabirohi/kidscribe-spl2-project, accessed 1 Nov. 2026.
6. Group 18, *README.md* and *ARCHITECTURE.md*, in the repository above.
7. M. Longcamp, M.-T. Zerbato-Poudou and J.-L. Velay, "The influence of writing practice on letter recognition in preschool children: A comparison between handwriting and typing," *Acta Psychologica*, vol. 119, no. 1, pp. 67–79, 2005.
8. K. H. James and L. Engelhardt, "The effects of handwriting experience on functional brain development in pre-literate children," *Trends in Neuroscience and Education*, vol. 1, no. 1, pp. 32–42, 2012.
9. J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: a comparative study," in *Proc. 15th Int. Conf. Pattern Recognition*, vol. 3, pp. 314–317, 2000.
10. Flutter documentation, https://docs.flutter.dev
11. Bloc library documentation, https://bloclibrary.dev
12. Google, ML Kit Digital Ink Recognition, https://developers.google.com/ml-kit/vision/digital-ink-recognition
13. Google, Firebase documentation (Authentication; Cloud Firestore security rules), https://firebase.google.com/docs
14. Microsoft, Fluent Emoji, MIT licence, https://github.com/microsoft/fluentui-emoji
15. Google Play, Families policies, https://play.google.com/about/families/
16. U.S. Federal Trade Commission, Children's Online Privacy Protection Rule, 16 CFR Part 312.
17. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.
18. United Nations, Sustainable Development Goal 4: Quality Education, https://sdgs.un.org/goals/goal4